

Dr. Zero: Self-Evolving Search Agents without Training Data

Zhenrui Yue^{1,2*}, Kartikeya Upasani¹, Xianjun Yang¹, Suyu Ge², Shaoliang Nie¹, Yuning Mao¹, Zhe Liu¹, Dong Wang²

¹Meta Superintelligence Labs, ²University of Illinois Urbana-Champaign

*Work done while at Meta

As high-quality data becomes increasingly difficult to obtain, self-evolution without curated training data has emerged as a promising paradigm. This approach allows large language models (LLMs) to autonomously generate and solve complex problems, thereby improving their reasoning capabilities. However, multi-turn search agents struggle in this setting due to limited question diversity and the substantial compute required for multi-step reasoning and tool use. In this work, we introduce Dr. Zero, a framework that enables search agents to effectively self-evolve without human-annotated training data, relying solely on an external search engine as their knowledge environment. In particular, we design a self-evolution feedback loop where a proposer generates structurally diverse questions to train a solver initialized from the same base model. As the solver evolves, it incentivizes the proposer to produce increasingly difficult yet solvable tasks, thus establishing an automated curriculum to refine both agents. To enhance training efficiency, we also introduce hop-grouped relative policy optimization (HRPO). This method clusters structurally similar questions to construct group-level baselines, effectively minimizing the sampling overhead in evaluating each query’s individual difficulty and solvability. Consequently, HRPO significantly reduces the compute requirements for proposer training and reward estimation without compromising performance or stability. Extensive experimental results demonstrate that Dr. Zero matches or surpasses fully supervised search agents on several question answering benchmarks, showing that strong agentic search and evidence-grounded reasoning can emerge solely through self-evolution.

Date: July 22, 2026

Code: <https://github.com/facebookresearch/drzero>

Correspondence: {zhenrui3, dwang24}@illinois.edu, kart@meta.com



1 Introduction

Self-evolving language agents have emerged as a promising paradigm to autonomously enhance model performance by iteratively proposing problems, bootstrapping solutions, and learning from prior experiences (Liu et al., 2025a; Zhang et al., 2025a). However, existing methods typically rely on extensive curated questions (prompts) to drive exploration, maintaining the need for the costly data curation process (Chen et al., 2024; Wang et al., 2025a). To overcome this bottleneck, an automated proposer can be employed to generate synthetic training data, thereby making self-evolution possible with minimal or even zero human input (Zhao et al., 2025; Chen et al., 2025). As shown in Figure 1, Huang et al. (2025a) design a proposer-solver co-evolution framework to iteratively bootstrap questions and rationales, thereby achieving meaningful performance gains without access to any human-annotated datasets.

Nevertheless, self-evolving language agents primarily focus on mathematical or specific reasoning tasks (Zhao et al., 2025; Huang et al., 2025a; Wang et al., 2025a). In these constrained domains, questions are often narrowly defined, allowing agents to achieve considerable gains even with limited or low-diversity training data (Zuo et al., 2025; Wang et al., 2025c). Yet for open-domain questions, self-evolving search agents remain substantially under-explored. While some approaches exist, they typically rely on human questions, extensive contexts or ground truth annotations (Jiang et al., 2025; Sun et al., 2025; Lu et al., 2025). To address this limitation, we focus on self-evolution without curated QA training data for learning search agents. Distinct

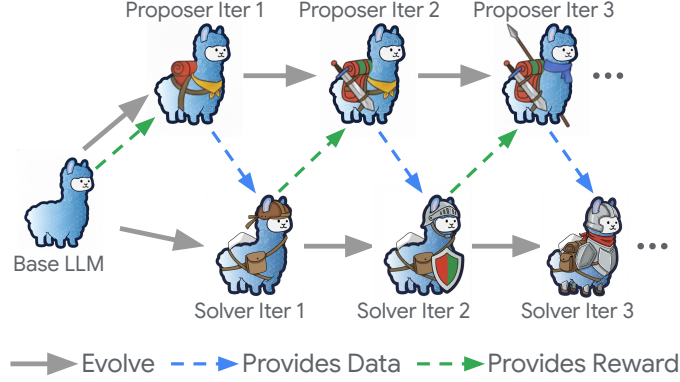


Figure 1 The self-evolving LLM training framework (Huang et al., 2025a) that iteratively trains a proposer model and a solver model with minimal supervision.

from prior work, we eliminate the need for human-written training questions or answer annotations, while using an external search engine and its indexed corpus as the knowledge environment for the agent’s self-evolution.

We analyze existing data-free frameworks (Huang et al., 2025a; Chen et al., 2025) and identify two primary limitations in proposer training: (1) the trained proposer suffers from limited diversity, exhibiting a bias toward simple, one-hop questions (see Section 2); and (2) their proposer objectives do not explicitly adapt query difficulty as the solver’s capabilities advance. As a result, such approaches yield moderate performance gains on trivial one-hop tasks but struggle to match supervised baselines on complex multi-hop queries (see Section 4). Moreover, the standard group relative policy optimization (GRPO) (Shao et al., 2024) significantly increases training compute in self-evolution as it requires nested sampling: generating multiple queries and subsequently producing multiple responses for each question. For instance, Chen et al. (2025) propose a unified model that acts as both proposer and solver for simple math and coding tasks. However, its reliance on nested sampling renders this approach computationally prohibitive for search agents and creates bottlenecks during complex reasoning and search interactions.

In this paper, we investigate self-evolving search agents without curated QA training data. We propose DeepResearch-Zero (Dr. Zero), a framework that leverages external search engines to improve both proposer and solver performance without human-written training questions or answers. Specifically, we introduce a multi-turn tool-use rollout pipeline that enables the trained proposer to improve question generation and produce complex, multi-hop questions. Furthermore, we design a difficulty-guided reward that incentivizes the proposer to utilize the search engine and generate challenging yet verifiable queries. As computing the reward involves sampling multiple reasoning trajectories from the solver, we also propose hop-grouped relative policy optimization (HRPO), which clusters structurally similar questions to provide a more robust baseline for advantage estimation. Thus, Dr. Zero avoids nested sampling by generating one question per seed prompt and estimating its baseline across questions with the same hop structure, rather than sampling multiple question candidates from each seed prompt as in standard GRPO. Combined with further training and inference optimizations, Dr. Zero achieves substantial gains compared to the base LLM. Additionally, through multiple training iterations, the model matches or even outperforms supervised RL baselines, showing that complex reasoning and search capabilities can be induced without human-annotated training questions in the evaluated short-form, verifiable QA setting. We summarize our contributions as follows:

1. We propose DeepResearch-Zero (Dr. Zero), a unified framework that leverages external search engines to autonomously improve both proposer and solver performance. Our approach integrates a refined multi-turn tool-use pipeline with a difficulty-guided reward to produce complex, multi-hop questions.
2. We introduce hop-grouped relative policy optimization (HRPO), a novel optimization method that clusters structurally similar questions to provide a robust group-level baseline for advantage estimation. This optimization ensures effective training while eliminating the need for expensive nested sampling in self-evolution.
3. We empirically demonstrate the effectiveness of Dr. Zero through extensive experiments, showing that

the proposed self-evolution paradigm yields significant performance gains. Without curated QA training data, our search agents match the aggregate performance of a fully supervised search agent at the 3B scale and outperform supervised baselines on several individual benchmarks.

2 Related Work

2.1 Reinforcement Learning

Reinforcement learning (RL) improves agent performance by learning from previous experience and maximizing cumulative rewards (Sutton et al., 1998). In the context of LLMs, RL is frequently implemented using policy gradient algorithms (Sutton et al., 1999; Ouyang et al., 2022). For example, actor-critic methods such as PPO employ a learned critic to estimate a value baseline, thereby reducing gradient variance (Mnih et al., 2016; Schulman et al., 2017). A simpler offline alternative is direct preference optimization (DPO) (Rafailov et al., 2023), which directly optimizes language models on pairwise preference data. Recently, group-based methods like GRPO have been adopted for their ability to construct low-variance baselines from multiple responses (Ahmadian et al., 2024; Shao et al., 2024; Hu, 2025). Building upon these algorithmic improvements, LLMs have demonstrated significant potential across diverse model architectures as well as reasoning-intensive tasks (Yu et al., 2025; Huang et al., 2025b; Zheng et al., 2025a).

2.2 Search-Augmented LLMs

Search and retrieval augmentation can improve language modeling by integrating external knowledge (Lewis et al., 2020; Guu et al., 2020). Furthermore, iteratively retrieving relevant documents can substantially enhance LLM performance on complex questions (Yoran et al., 2024; Yue et al., 2024; Wang et al., 2025b). A notable example is IRCoT, where Trivedi et al. (2023) exploit multi-step retrieval to optimize answer accuracy on knowledge-intensive tasks. In agentic LLMs, search engines can be incorporated as optional tools to enable adaptive retrieval and reasoning, leading to enhanced multi-hop reasoning performance (Jin et al., 2025; Zheng et al., 2025b; Zhang et al., 2025b). Nevertheless, existing methods primarily focus on supervised, verifiable settings and rely on extensive human queries and annotations for training. Concurrent with our work, Lu et al. (2025) propose using self-play to improve search-augmented LLMs. However, their method still depends on ground truth labels and numerous human-written examples. In contrast, we use no curated QA training data and rely on external search as the knowledge environment for self-improvement.

2.3 Self-Evolving LLMs

Self-evolving LLMs autonomously enhance model capabilities by iteratively generating and learning from their own experiences (Pang et al., 2024; Liu et al., 2025a). Early approaches utilize self-play mechanisms where the model acts as both the generator and the evaluator to refine its policy without human annotations (OpenAI et al., 2021; Chen et al., 2024; Wu et al., 2024). For instance, self-rewarding LLMs (Yuan et al., 2024) employ iterative training loops where the model judges its own outputs to construct preference data for optimization. To further eliminate the reliance on external prompts, recent works have introduced data-free self-evolution frameworks (Zhao et al., 2025; Chen et al., 2025; Liu et al., 2025b; Xia et al., 2025). Notably, R-Zero (Huang et al., 2025a) adopts a proposer-solver design to co-evolve their performance from scratch, effectively creating a self-improving curriculum. Although promising, existing methods are computationally intensive for multi-turn search agents and often underperform on open-domain questions due to the lack of diverse data and tool integration. To bridge this gap, we introduce Dr. Zero, an efficient self-evolution framework without curated QA training data that enables search agents to match or even exceed the performance of strong supervised baselines.

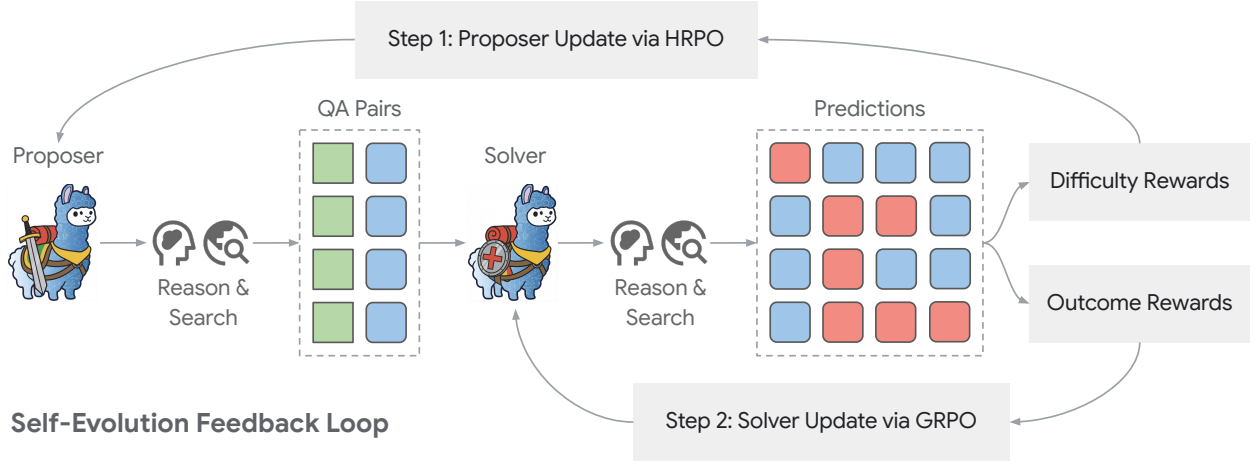


Figure 2 The Dr. Zero self-evolution feedback loop. Guided by solver feedback, the proposer synthesizes verifiable and challenging queries, continuously enhancing the solver’s search and reasoning capabilities.

3 Methodology

3.1 Setup

We employ a proposer-solver self-evolution framework where both models function as search agents capable of leveraging external knowledge. Equipped with the external search engine $\mathcal{R}$, the proposer π_θ and the solver π_ϕ are trained to maximize their respective expected rewards:

$$\begin{aligned} \text{Proposer : } & \mathbb{E}_{(x,y) \sim \pi_\theta(\cdot|\mathcal{R}), \{\hat{y}_i\}_{i=1}^n \sim \pi_\phi(\cdot|x,\mathcal{R})} [r(y, \{\hat{y}_i\}_{i=1}^n)], \\ \text{Solver : } & \mathbb{E}_{(x,y) \sim \pi_\theta(\cdot|\mathcal{R}), \hat{y} \sim \pi_\phi(\cdot|x,\mathcal{R})} [\mathbb{I}(y = \hat{y})], \end{aligned} \quad (1)$$

where r denotes the proposer reward and $\mathbb{I}$ is the indicator function. In contrast to the solver’s simple outcome-based reward, the proposer reward is defined over the distribution of predicted answers (i.e., $\{\hat{y}_i\}_{i=1}^n$). If all predictions are correct, the question is considered trivial, whereas if none are correct, the question is likely too difficult for the solver. To enable self-evolution of π_θ and π_ϕ , we iteratively optimize both components in a symbiotic loop: the proposer learns to synthesize diverse and challenging questions, while the solver enhances its reasoning abilities by learning from these questions. The improved solver performance subsequently encourages the proposer to generate increasingly complex queries, forming a continuously evolving curriculum (see Figure 2). We initialize both models from the same base LLM and rely exclusively on the search tool ($\mathcal{R}$) for external knowledge. To adhere to our setting without curated QA training data, we avoid utilizing any demonstrations, human-written questions or annotated answers in our framework.

3.2 Proposer Training

Existing self-evolution methods primarily target specialized domains (e.g., math, coding) to enhance LLM performance without external data (Zhao et al., 2025; Huang et al., 2025a; Chen et al., 2025). However, for open-domain question answering, we find that such methods tend to generate structurally homogeneous one-hop queries, which limits solver performance gains to simple reasoning questions. Even when equipped with a multi-turn search tool, the solver yields only marginal improvements on multi-hop queries (see Section 4). Furthermore, since the proposer reward necessitates multiple solver predictions to assess query difficulty and solvability, optimizing the proposer with a standard RL algorithm like GRPO becomes highly inefficient (Shao et al., 2024). Generating m candidate QA pairs from each seed prompt and evaluating each with n solver predictions requires $m + mn = m(n + 1)$ rollouts. In our implementation, this corresponds to 4 proposer rollouts and 4×4 solver rollouts (20 total). Combined with the high latency of multi-turn rollouts, this scaling bottleneck renders existing approaches impractical for self-evolving agents that require complex tool interactions.

Motivated by these limitations, we propose hop-grouped relative policy optimization (HRPO) to train the proposer model. Instead of sampling multiple responses for a single seed prompt, HRPO calculates advantages by grouping structurally similar questions across prompts. Specifically, we cluster the generated QA pairs by their cross-hop complexity, denoted by the number of hops $h \in \mathcal{H}$. Questions with fewer hops are typically simpler, whereas higher-hop questions demand extensive search and multi-step reasoning. Here, hop count is used only as a structural grouping variable; empirical question difficulty is determined by the solver pass rate in Equation (4). This hop-specific normalization of returns produces low-variance advantage estimates while avoiding the computational cost of sampling multiple candidate questions per seed prompt. With five solver predictions, HRPO requires one proposer rollout and five solver rollouts (6 total). HRPO can be formulated with:

$$\mathcal{J}(\theta) = \mathbb{E}_{\{(x_i, y_i) \sim \pi_\theta(\cdot | \mathcal{R}), \{\hat{y}_{i,k}\}_{k=1}^n \sim \pi_\phi(\cdot | x_i, \mathcal{R})\}_{i=1}^N} \left[\frac{1}{N} \sum_{h \in \mathcal{H}} \sum_{i \in \mathcal{I}_h} \log \pi_\theta(x_i, y_i | \mathcal{R}) A_{i,h} \right] - \beta \mathbb{D}_{KL}, \quad (2)$$

where N denotes the size of the sampled batch, and β is the hyperparameter controlling the KL regularizer. $A_{i,h}$ denotes the advantage of the generated QA pair (x_i, y_i) , computed by standardizing the solver’s reward scores over all h -hop questions:

$$A_{i,h} = \frac{r_i - \mathbb{E}_{j \in \mathcal{I}_h} [r_j]}{\sqrt{\text{Var}_{j \in \mathcal{I}_h} [r_j]} + \delta}. \quad (3)$$

For optimal proposer performance and training efficiency, we adopt a strictly on-policy framework and omit ratio clipping. While single-response methods like REINFORCE++ reduce sampling costs, we find that a global baseline becomes unstable when processing diverse query structures. This mismatch induces high variance in policy gradients, frequently leading to training failures. In contrast, HRPO mitigates this issue by computing relative advantages among structurally aligned trajectories.

To utilize solver signals for training the proposer π_θ , we design a specialized reward function to encourage both verifiability (the task must be solvable) and difficulty (the task must not be trivial). We leverage the solver’s pass rate on the generated questions as a proxy for these properties. Let k denote the number of correct solutions out of n sampled attempts; we penalize instances where the solver either fails completely ($k = 0$) or succeeds trivially ($k = n$), incentivizing the generation of questions that maximize:

$$r(y, \{\hat{y}_i\}_{i=1}^n) = \mathbb{I}(0 < k < n) \frac{n-k}{n-1} + r^f, \text{ with } k = \sum_{i=1}^n \mathbb{I}(y = \hat{y}_i), \quad (4)$$

here, the reward is maximized when exactly one solution is correct and decays linearly as the number of correct predictions increases. We additionally impose a format reward r^f to mitigate structural degradation during complex generation. This enables the proposer to effectively learn to interleave reasoning with search, yielding QA pairs that are both well-formed and challenging. The proposer prompt also requires each hop and final answer to be supported by the seed or retrieved passages (see Figure 5). These grounding requirements and the absence of difficulty reward for zero-pass cases reduce, but do not eliminate, incorrect or ambiguous synthetic answers; richer signals such as retrieval uncertainty or answer ambiguity could be incorporated for less-verifiable settings.

3.3 Solver Training

For solver training, we sample data pairs (x, y) from the proposer π_θ and optimize π_ϕ via group relative policy optimization (GRPO) (Shao et al., 2024). By computing advantages from the empirical group statistics, GRPO reinforces valid trajectories and refines the model’s search and reasoning capabilities without requiring a separate value function:

$$\mathcal{J}(\phi) = \mathbb{E}_{(x, y) \sim \pi_\theta(\cdot | \mathcal{R}), \{\hat{y}_i\}_{i=1}^n \sim \pi_{\phi_{\text{old}}}(\cdot | x, \mathcal{R})} \left[\frac{1}{n} \sum_{i=1}^n \min \left(\frac{\pi_\phi(\hat{y}_i | x, \mathcal{R})}{\pi_{\phi_{\text{old}}}(\hat{y}_i | x, \mathcal{R})} A_i, \text{clip} \left(\frac{\pi_\phi(\hat{y}_i | x, \mathcal{R})}{\pi_{\phi_{\text{old}}}(\hat{y}_i | x, \mathcal{R})}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) \right] - \beta \mathbb{D}_{KL}, \quad (5)$$

where the advantages are computed via reward standardization (i.e., $A_i = \frac{\mathbb{I}(y=\hat{y}_i) - \text{mean}(\{\mathbb{I}(y=\hat{y}_i)\}_{i=1}^n)}{\text{std}(\{\mathbb{I}(y=\hat{y}_i)\}_{i=1}^n) + \delta}$). The optimization is driven by an outcome-based reward that solely evaluates the correctness of final predictions

Table 1 Main results of Dr. Zero against few-shot/supervised methods. We mark the best performance in bold and underline the second-best results. Unlike baselines that require curated datasets or extensive demonstrations, Dr. Zero exploits self-evolution to *match or even outperform supervised search agents without curated QA training data*.

	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Average
Qwen2.5-3B-Instruct								
Prompting	0.106	0.288	0.108	0.149	0.244	0.020	0.024	0.134
IRCoT	0.111	0.312	0.200	0.164	0.171	0.067	0.240	0.181
Search-o1	0.238	0.472	0.262	0.221	0.218	0.054	0.320	0.255
RAG	<u>0.348</u>	<u>0.544</u>	<u>0.387</u>	0.255	0.226	0.047	0.080	0.270
SFT	<u>0.249</u>	<u>0.292</u>	<u>0.104</u>	0.186	0.248	0.044	0.112	0.176
R1-Instruct	0.210	0.449	0.171	0.208	0.275	0.060	0.192	0.224
Search-R1	0.323	0.537	0.364	0.308	0.336	0.105	<u>0.315</u>	0.327
Dr. Zero	0.397	0.572	0.431	<u>0.298</u>	<u>0.291</u>	<u>0.091</u>	0.200	<u>0.326</u>
Qwen2.5-7B-Instruct								
Prompting	0.134	0.408	0.140	0.183	0.250	0.031	0.120	0.181
IRCoT	0.224	0.478	0.301	0.133	0.149	0.072	0.224	0.239
Search-o1	0.151	0.443	0.131	0.187	0.176	0.058	0.296	0.206
RAG	0.349	0.585	0.392	0.299	0.235	0.058	0.208	0.304
SFT	0.318	0.354	0.121	0.217	0.259	0.066	0.112	0.207
R1-Instruct	0.270	0.537	0.199	0.237	0.292	0.072	0.293	0.271
Search-R1	0.397	<u>0.606</u>	<u>0.404</u>	0.380	<u>0.326</u>	0.168	0.408	0.384
Dr. Zero	0.406	0.608	0.416	<u>0.362</u>	0.347	<u>0.104</u>	<u>0.360</u>	<u>0.372</u>

against the synthesized ground truth y . With increasingly complex queries from the proposer, the solver is motivated to refine its search and reasoning capabilities. Such interactions create a dynamic curriculum that supports solver improvement across diverse problem domains without curated QA training data. The solver’s progress, in turn, encourages the proposer to synthesize more complex cases, establishing a feedback loop that can expand both agents’ capabilities.

3.4 The Self-Evolving Dr. Zero

In summary, we introduce Dr. Zero, a scalable and effective framework that leverages self-evolution without curated QA training data to iteratively enhance both the proposer and solver (see Figure 2). In each iteration, the proposer π_θ synthesizes a batch of QA pairs with heterogeneous hop structures. Utilizing solver feedback, the proposer is optimized via HRPO to produce verifiable, diverse and challenging queries. Meanwhile, the solver leverages the generated data through GRPO to refine its search and reasoning capabilities. This alternating optimization loop creates a symbiotic feedback mechanism: as the solver improves, simple queries yield diminishing rewards, encouraging the proposer to explore more complex reasoning paths to maximize its returns. Conversely, increasingly difficult questions help sustain useful training signals, allowing the solver to continue refining its reasoning skills. Both models are initialized from the same base LLM and evolve without human-written training questions or annotated answers, relying on the external search environment to drive their performance improvements.

4 Experiments

4.1 Experiment Settings

Datasets & Models. For evaluation, we experiment on multiple open-domain question answering benchmarks, including three one-hop datasets Natural Questions (NQ) (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022); and four multi-hop datasets HotpotQA (Yang et al., 2018),

Table 2 Performance comparison between Dr. Zero and data-free baseline methods.

	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Average
Qwen2.5-3B-Instruct								
SQLM*	0.264	0.432	0.258	0.226	0.238	0.060	0.158	0.233
R-Zero*	0.389	0.513	0.370	0.243	0.128	0.052	0.096	0.256
Dr. Zero	0.397	0.572	0.431	0.298	0.291	0.091	0.200	0.326

2WikiMultihopQA (2WikiMQA) (Ho et al., 2020), MuSiQue (Trivedi et al., 2022) and Bamboogle (Press et al., 2023). These datasets cover diverse search and reasoning challenges, ensuring a comprehensive evaluation of Dr. Zero across both single-turn and multi-hop scenarios. In our experiments, we use Qwen2.5 3B/7B Instruct as base LLMs for both baseline methods and Dr. Zero.

Baseline & Evaluation. To demonstrate the efficacy of Dr. Zero, we evaluate it against several *few-shot* and *supervised* baseline search agents. Few-shot baselines include standard prompting, IRCOT (Trivedi et al., 2023), Search-o1 (Li et al., 2025) and retrieval augmented generation (RAG) (Lewis et al., 2020). Supervised baselines consist of supervised fine-tuning (SFT), RL-based fine-tuning without search (R1) (Guo et al., 2025) and the RL-based search agent Search-R1 (Jin et al., 2025). All models are evaluated using exact match with identical search engine (E5 base) and corpus settings (English Wikipedia dump). *Unlike the baselines, Dr. Zero uses neither human-annotated QA training data nor demonstrations; the shared Wikipedia corpus and search engine serve as its external knowledge environment.* Further implementation details are provided in Section .1.

4.2 Experiment Results

Overall Performance. We first discuss the main evaluation results as reported in Table 1. Based on the presented results, we draw several key observations: (1) *Overall Performance*: Dr. Zero is competitive with the strongest supervised baselines across both single-hop and multi-hop benchmarks, and exceeds them on several individual tasks. These results demonstrate that Dr. Zero effectively extends autonomous self-evolution to search agents without curated QA training data. (2) *Superiority over Few-Shot Baselines*: Dr. Zero substantially outperforms few-shot methods across nearly every benchmark. For instance, using Qwen2.5-3B on NQ, Dr. Zero achieves 0.397 EM, significantly outperforming few-shot prompting (0.106), IRCOT (0.111) and Search-o1 (0.238). Unlike such baselines, our iterative training loop dynamically refines search capabilities across all tasks. (3) *Parity with Supervised Search Agents*: Remarkably, Dr. Zero excels *without human-annotated QA training data*. On single-hop tasks (i.e., NQ, TriviaQA and PopQA), our 3B model outperforms the supervised Search-R1 by 22.9%, 6.5% and 18.4% respectively. In complex multi-hop scenarios, the 7B variant achieves roughly 90% of Search-R1’s performance and even outperforms it on the challenging 2WikiMQA. (4) *Scaling and Self-Evolution*: As the base model scales from 3B to 7B, Dr. Zero exhibits robust gains, particularly in multi-hop reasoning. By utilizing HRPO and a difficulty-guided reward, our framework generates a high-quality curriculum that supports gains across the initial self-evolution iterations. Overall, our findings demonstrate that Dr. Zero effectively leverages the proposer-solver dynamics to enhance agentic search and evidence-grounded answering across diverse benchmarks; our evaluation does not separately isolate these gains from reasoning over retrieved passages.

Comparison to Data-Free Baselines. We further compare Dr. Zero against existing data-free methods, specifically self-questioning language models (SQLM) and self-evolving reasoning LLMs (R-Zero) (Chen et al., 2025; Huang et al., 2025a). *To ensure a fair comparison, we augment baselines with multi-turn reasoning and search capabilities, denoting them SQLM* and R-Zero**. Experiment results on the 3B backbone are reported in Table 2. From these results, we observe the following: (1) Dr. Zero consistently performs the best across all tasks, confirming that our proposer and training pipeline are highly effective for data-free search agents. It exceeds SQLM* and R-Zero* by an average of 39.9% and 27.3% respectively. (2) Unlike the zero-variance filtering used by SQLM*, our pass-rate reward explicitly adapts to the current solver: questions for which all solver attempts are correct or incorrect receive no difficulty reward, while partially solved questions are favored. This design provides a direct signal for producing challenging yet solvable questions. (3) Dr. Zero achieves superior

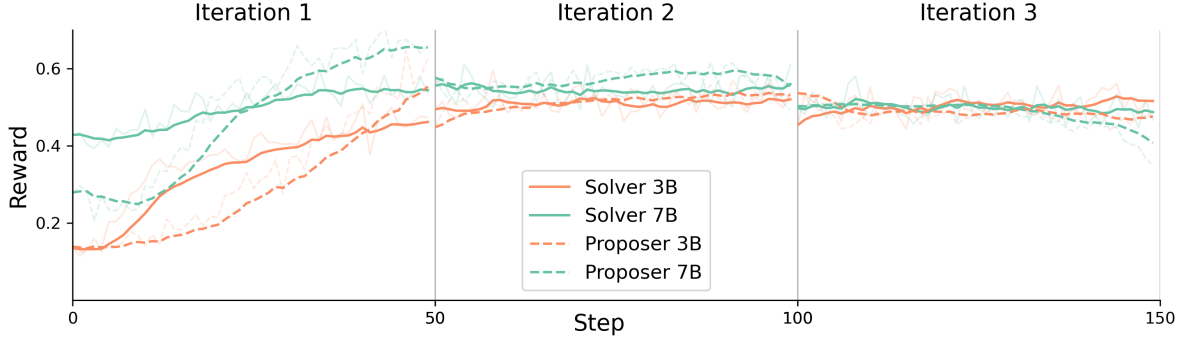


Figure 3 Iterative reward dynamics of the proposer and solver in Dr. Zero. The downward shifts in baseline rewards across iterations reflect the co-evolution of the models; as one model strengthens, it naturally lowers the initial reward floor for the other, thereby driving further self-improvement through reinforcement learning.

Table 3 Learning dynamics of Dr. Zero with increasing iterations, we mark the best performance in bold.

	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Average
Qwen2.5-3B-Instruct								
Dr. Zero Iter 1	0.381	0.526	0.392	0.284	0.243	0.084	0.216	0.304
Dr. Zero Iter 2	0.401	0.563	0.408	0.289	0.255	0.102	0.216	0.319
Dr. Zero Iter 3	0.397	0.572	0.431	0.298	0.291	0.091	0.200	0.326
Qwen2.5-7B-Instruct								
Dr. Zero Iter 1	0.392	0.597	0.395	0.347	0.361	0.108	0.360	0.366
Dr. Zero Iter 2	0.406	0.608	0.416	0.362	0.347	0.104	0.360	0.372
Dr. Zero Iter 3	0.416	0.608	0.412	0.352	0.319	0.107	0.320	0.360

performance and higher efficiency through the proposed HRPO and reward formulation. Specifically, hop-based clustering significantly reduces proposer training costs without compromising performance. Correspondingly, Dr. Zero yields an average relative gain of 83.3% over R-Zero* across the four multi-hop benchmarks. Overall, by integrating a multi-turn reasoning-search framework with difficulty-guided rewards and the proposed HRPO, Dr. Zero demonstrates clear advantages over existing data-free methods.

Training Dynamics. To better understand the self-evolving dynamics of Dr. Zero, we investigate the performance and rewards across iterations, with detailed reward curves and performance metrics summarized in Figure 3 and Table 3. These results provide several key observations: (1) We observe a steady upward trend in performance and rewards at the start of training. Using exclusively synthetic data from the proposer, both the 3B and 7B solvers rapidly reach a performance peak within approximately 50 steps, suggesting that the most significant gains are realized during this initial phase. (2) Following the first training phase, both the proposer and solver show substantial gains in search and reasoning. The second iteration yields consistent improvements across both solver sizes, with average gains of 4.93% and 1.64% across benchmarks. This confirms the continuous self-evolutionary trend of Dr. Zero. (3) After the second iteration, results vary by model size. The 3B model shows a modest 2.2% increase, but the 7B model drops slightly from 0.372 to 0.360, indicating a performance plateau. Beyond this point, additional iterations for either model size yielded only marginal or no further improvements. (4) We identify several training failure modes, with the most common of which stemming from inconsistent token IDs across multi-turn search and reasoning steps. Interestingly, the 7B model exhibits this trend more frequently (e.g., 7B proposer iteration 3 in Figure 3), leading to increased training instability compared to the 3B variant. We also present a comparison with GRPO in Section 2. In summary, the results highlight the effectiveness of the proposer-solver interplay while identifying the technical constraints that limit indefinite self-evolution. These dynamics validate the design of our Dr. Zero framework, illustrating that autonomous self-evolution successfully replaces supervision for developing multi-turn reasoning and search capabilities.

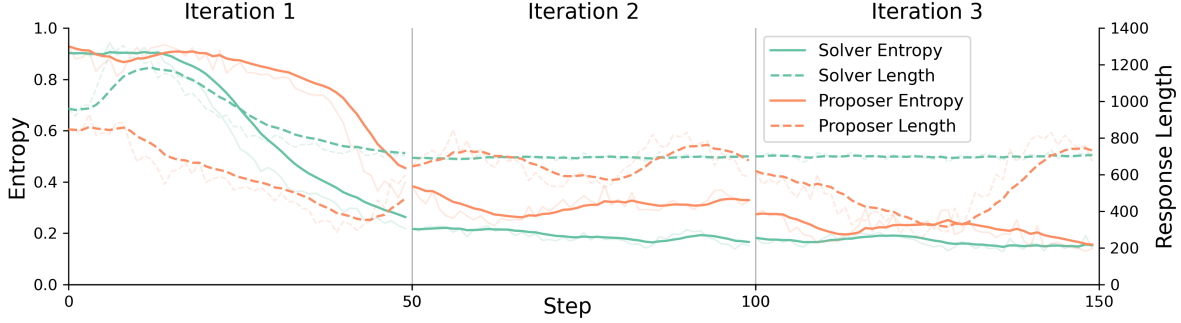


Figure 4 Averaged entropy values and response lengths of Dr. Zero 3B during training.

Table 4 Performance of Dr. Zero trained with different distributions of generated questions.

	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Average
Qwen2.5-3B-Instruct								
Ratio 1:1:1:1	0.402	0.549	0.394	0.297	0.287	0.090	0.176	0.314
Ratio 2:1:1:1	0.398	0.560	0.408	0.287	0.268	0.096	0.192	0.316
Ratio 4:3:2:1	0.397	0.572	0.431	0.298	0.291	0.091	0.200	0.326
Qwen2.5-7B-Instruct								
Ratio 1:1:1:1	0.392	0.591	0.396	0.363	0.352	0.116	0.368	0.368
Ratio 2:1:1:1	0.398	0.585	0.396	0.365	0.346	0.109	0.384	0.369
Ratio 4:3:2:1	0.406	0.608	0.416	0.362	0.347	0.104	0.360	0.372

Response Length and Entropy. Aside from performance, we investigate response entropy and length throughout the training process. Here, we focus on Dr. Zero 3B variant and visualize the average entropy values and response length in Figure 4, leading to several noteworthy observations: (1) For the solver, both entropy and response length decrease gradually before stabilizing toward the end of the first iteration. This trend indicates that the solver acquires its core search and reasoning capabilities early on, followed by steady, incremental refinements in subsequent iterations. (2) While the proposer follows an initial trend similar to the solver, its entropy and length fluctuate in later iterations. These variations indicate that the proposer maintains generation diversity, continuously exploring different paths to present the solver with increasingly complex challenges. (3) Compared to trajectory lengths and the proposer entropy, the solver’s entropy values are more stable; they decrease drastically and stabilize at a low level. This trend suggests that the model learns quickly and becomes more confident as its reasoning capabilities improve. Together, these trends confirm that the proposer-solver interplay creates a dynamic learning environment: the solver achieves rapid and stable convergence, while the proposer preserves the generation diversity to ensure a broader spectrum of challenging, high-quality questions.

Different Hop Ratios. Finally, we inspect how question composition affects the overall performance of Dr. Zero. Specifically, we vary the distribution of synthetic questions from the proposer, testing ratios of 1/2/3/4-hop questions at 1:1:1:1, 2:1:1:1 and our default 4:3:2:1. These comparative results are detailed in Table 4. Interestingly, the 3B model does not exhibit performance gains as the proportion of multi-hop questions increases. The optimal multi-hop performance is achieved using the 4:3:2:1 ratio, yielding an average 0.220 EM. This suggests that for smaller base models, strengthening fundamental search capabilities (even via single-hop queries) can more effectively improve performance than focusing exclusively on complex tasks. In contrast, the 7B variant of Dr. Zero achieves its highest overall average with the default 4:3:2:1 composition, while the other compositions remain slightly better on some complex benchmarks. This indicates that performance depends on balancing questions across hop levels rather than simply maximizing the proportion of multi-hop questions. In summary, both model sizes benefit from a mixed curriculum, while the most effective balance varies by model scale and benchmark.

5 Conclusion

We introduced Dr. Zero, a self-evolution framework that enhances the reasoning and search capabilities of language agents without curated QA training data. By utilizing an iterative proposer-solver training paradigm, Dr. Zero autonomously generates diverse and increasingly challenging short-form, verifiable open-domain questions without human-written questions or annotated answers. In addition, the proposed HRPO effectively addresses the computational bottlenecks of multi-turn tool use, enabling efficient training by clustering structurally similar queries to estimate advantages. Experimental results demonstrate that Dr. Zero is competitive with supervised search agents and surpasses them on several question answering benchmarks. These findings validate the potential of the self-evolving Dr. Zero as a powerful paradigm for developing advanced search agents in data-scarce environments. Future work will focus on extending the stability of self-evolution to overcome performance plateaus and prevent entropy collapse in larger models. Furthermore, we plan to safeguard the self-evolution process against reward hacking and bias amplification, aiming to develop robust learning frameworks that maintain the integrity and reliability of the feedback loop even in the absence of human supervision.

References

- Arash Ahmadian, Chris Cremer, Matthias Gall , Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet  st n, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.
- Lili Chen, Mihir Prabhudesai, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Self-questioning language models. *arXiv preprint arXiv:2508.03682*, 2025.
- Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, 2020.
- Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.
- Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*, 2025a.
- Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin. Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*, 2025b.
- Pengcheng Jiang, Xueqiang Xu, Jiacheng Lin, Jinfeng Xiao, Zifeng Wang, Jimeng Sun, and Jiawei Han. s3: You don’t need that much data to train a search agent via rl. *arXiv preprint arXiv:2505.14146*, 2025.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, 2017.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K ttler, Mike Lewis, Wen-tau Yih, Tim Rock schel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366*, 2025.
- Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, et al. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *arXiv preprint arXiv:2506.24119*, 2025a.
- Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. Spice: Self-play in corpus environments improves reasoning. *arXiv preprint arXiv:2510.24684*, 2025b.
- Hongliang Lu, Yuhang Wen, Pengyu Cheng, Ruijin Ding, Haotian Xu, Jiaqi Guo, Chutian Wang, Haonan Chen, Xiaoxi Jiang, and Guanjin Jiang. Search self-play: Pushing the frontier of agent capability without supervision. *arXiv preprint arXiv:2510.18821*, 2025.

- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.
- OpenAI OpenAI, Matthias Plappert, Raul Sampedro, Tao Xu, Ilge Akkaya, Vineet Kosaraju, Peter Welinder, Ruben D’Sa, Arthur Petron, Henrique P d O Pinto, et al. Asymmetric self-play for automatic goal discovery in robotic manipulation. *arXiv preprint arXiv:2101.04882*, 2021.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- Richard Yuanzhe Pang, Weizhe Yuan, He He, Kyunghyun Cho, Sainbayar Sukhbaatar, and Jason Weston. Iterative reasoning preference optimization. *Advances in Neural Information Processing Systems*, 37:116617–116637, 2024.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, 2023.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36:53728–53741, 2023.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. Zeroset: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588*, 2025.
- Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*. MIT press Cambridge, 1998.
- Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10014–10037, 2023.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- Shaobo Wang, Zhengbo Jiao, Zifan Zhang, Yilang Peng, Xu Ze, Boyu Yang, Wei Wang, Hu Wei, and Linfeng Zhang. Socratic-zero: Bootstrapping reasoning via data-free agent co-evolution. *arXiv preprint arXiv:2509.24726*, 2025a.
- Shu Wang, Yixiang Fang, Yingli Zhou, Xilin Liu, and Yuchi Ma. Archrag: Attributed community-based hierarchical retrieval-augmented generation. *arXiv preprint arXiv:2502.09891*, 2025b.
- Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Liyuan Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, et al. Reinforcement learning for reasoning in large language models with one training example. *arXiv preprint arXiv:2504.20571*, 2025c.
- Yue Wu, Zhiqing Sun, Huizhuo Yuan, Kaixuan Ji, Yiming Yang, and Quanquan Gu. Self-play preference optimization for language model alignment. *arXiv preprint arXiv:2405.00675*, 2024.
- Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*, 2025.

- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.
- Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. Making retrieval-augmented language models robust to irrelevant context. In *The Twelfth International Conference on Learning Representations*, 2024.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason E Weston. Self-rewarding language models. In *Forty-first International Conference on Machine Learning*, 2024.
- Zhenrui Yue, Honglei Zhuang, Aijun Bai, Kai Hui, Rolf Jagerman, Hansi Zeng, Zhen Qin, Dong Wang, Xuanhui Wang, and Michael Bendersky. Inference scaling for long-context retrieval augmented generation. *arXiv preprint arXiv:2410.04343*, 2024.
- Ding-Chu Zhang, Yida Zhao, Jialong Wu, Liwen Zhang, Baixuan Li, Wenbiao Yin, Yong Jiang, Yu-Feng Li, Kewei Tu, Pengjun Xie, et al. Evolvesearch: An iterative self-evolving search agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13134–13147, 2025a.
- Weizhi Zhang, Yangning Li, Yuanchen Bei, Junyu Luo, Guancheng Wan, Liangwei Yang, Chenxuan Xie, Yuyao Yang, Wei-Chieh Huang, Chunyu Miao, et al. From web search towards agentic deep research: Incentivizing search with reasoning agents. *arXiv preprint arXiv:2506.18959*, 2025b.
- Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025a.
- Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. *arXiv preprint arXiv:2504.03160*, 2025b.
- Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, et al. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*, 2025.

Appendix

.1 Implementation

In our experiments, we implement Dr.Zero as an alternating optimization of the proposer and solver, as illustrated in Figure 1. We start by training the proposer and utilizing the base model as the generative reward Yang et al. (2024). Specifically, we rollout one response per prompt and extract the corresponding question and answer. These serve as inputs to the solver to compute the reward defined in Equation (4). Finally, this reward is employed by HRPO to update the proposer model, as described in Equation (2). We observe that the reward saturates within approximately 50 steps due to our efficient design. Therefore, we train the proposer for 50 steps, generate QA data on the corresponding prompts, and subsequently utilize this data to train the solver for 50 steps via GRPO Shao et al. (2024). In our experiments, the proposer is configured with a default generation ratio of 4:3:2:1 for 1-, 2-, 3-, and 4-hop questions respectively. Under this setting, the solver performance typically peaks after 2 to 3 self-evolution iterations, with further training yielding only marginal improvements. Therefore, we limit training to 3 iterations (150 steps per model), which is significantly fewer than baselines like R1 and Search-R1. For further hyperparameters, we conduct a minimal search over the maximum gradient norm and KL divergence coefficient to maintain training stability. The hyperparameters are held constant in our experiments, with full details reported in Table 5 and Table 6.

Table 5 Proposer (HRPO) hyperparameter settings.

Algorithm	HRPO
Steps	50
Optimizer	AdamW
Optimizer Momentum	$\beta_1, \beta_2 = 0.9, 0.999$
Warmup Ratio	0.03
Weight Decay	0.01
Learning Rate	5e-7, 1e-6
Max Gradient Norm	0.1, 1.0
Group size in HRPO	1
Reward size in HRPO	5
KL-Div in HRPO	0
Total Train Batch Size	256
LR Scheduler	Constant with Warmup
Precision (WA)	BF16-mixed
Max Turn in Rollout	5
Max Sequence Length	4096

For the format reward in HRPO, we define four requirements: (1) adherence to the `<think>...</think>` structure; (2) valid tool usage, including correct tool call and arguments; (3) an extractable question enclosed in `<question>...</question>` tags; and (4) an extractable answer within `<answer>...</answer>` tags. These components are computed individually with a lower bound of 0 and sum to a maximum total of 0.5. This stands in contrast to the difficulty score, which ranges from 0 to 1 (see Equation (4)). We detail the tool instructions and prompts for the proposer and solver in Figure 5 and Figure 6, respectively, with qualitative examples available in Section .3. For the external search engine, we configure it by indexing the corpus and embedding queries using the E5 model Wang et al. (2022) following Yue et al. (2024); Jin et al. (2025). During inference, we perform an approximate nearest neighbor (ANN) search to retrieve the top-3 documents. These passages are subsequently formatted and returned as the tool response. Finally, for both training and evaluation, we utilize exact match to calculate the instance-level score.

.2 Additional Results

We first provide a detailed quantitative breakdown of the computational efficiency of HRPO compared to standard GRPO. As noted in the main text, standard GRPO relies on asynchronous, nested rollouts, which do not scale efficiently to larger LLMs such as our 7B model. To highlight the efficiency gains, we measured the

Table 6 Solver (GRPO) hyperparameter settings.

Algorithm	GRPO
Steps	50
Optimizer	AdamW
Optimizer Momentum	$\beta_1, \beta_2 = 0.9, 0.999$
Warmup Ratio	0.03
Weight Decay	0.01
Learning Rate	1e-6
Max Gradient Norm	0.1, 1.0
Group size in GRPO	5
KL-Div in GRPO	0, 0.001
ϵ in GRPO	0.2
Total Train Batch Size	256
LR Scheduler	Constant with Warmup
Precision (WA)	BF16-mixed
Max Turn in Rollout	5
Max Sequence Length	3072

Table 7 Ablation results of Dr. Zero.

	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle
Qwen2.5-3B-Instruct							
Dr. Zero (50 steps)	0.381	0.526	0.392	0.284	0.243	0.084	0.216
w/ 100 train steps	0.379	0.552	0.425	0.281	0.243	0.064	0.168
w/o format reward	0.365	0.501	0.350	0.287	0.272	0.056	0.192
w/ parabolic reward	0.388	0.541	0.408	0.279	0.246	0.058	0.144
w/o initial document	0.273	0.461	0.239	0.239	0.241	0.099	0.163

training time for the 3B model (referenced in Table 9) using a single 8xH100 node. HRPO requires significantly fewer rollouts per iteration compared to standard GRPO with nested sampling. This reduction translates directly to a substantial decrease in both wall-clock time and total GPU-hours per iteration, alongside an improvement in average GPU utilization¹.

We provide additional results validating the design choices behind Dr. Zero and HRPO. Specifically, we assess the effectiveness of the proposer within Dr. Zero by ablating key components, such as the format reward and the difficulty-based reward. To accommodate computational constraints, we limit the training of the 3B backbone to a single iteration while selectively removing components from the full framework. We examine the following variations: (1) extending training to 100 steps; (2) removing the format reward; (3) employing a parabolic reward in the proposer (which peaks when solver accuracy is approximately 50%); and (4) omitting the initial document. The ablation results are reported in Table 7, based on which we observe: (1) increasing the training duration from 50 to 100 steps results in comparable or slightly degraded average performance (0.304 vs. 0.302), suggesting that Dr. Zero converges efficiently and does not benefit from prolonged training in each self-evolution iteration. (2) Removing the format reward results in an average performance drop from 0.304 to 0.289, validating its critical role in guiding the proposer to conduct effective search and reasoning while maintaining structural validity. (3) The default difficulty-based reward outperforms the parabolic design. This indicates that our proposed reward shaping provides more effective signals for generating challenging queries compared to a simple parabolic objective. (4) The most significant performance drop occurs when the initial document is removed (average score falls to 0.245). This indicates that the initial context is crucial for the model to generate diverse synthetic data. Overall, the results demonstrate that each component of Dr. Zero plays a critical role in guiding the proposer to generate structurally valid and challenging queries, validating our integrated design as an effective framework for self-evolution without curated QA training data.

¹The solver training remains identical for both methods, completing in approximately 3 hours.

Table 8 Comparison between HRPO and GRPO.

	HRPO	GRPO
	Qwen2.5-3B-Instruct	
NQ	0.397	0.361
TriviaQA	0.572	0.548
PopQA	0.431	0.377
HotpotQA	0.298	0.303
2WikiMQA	0.291	0.279
MuSiQue	0.091	0.100
Bamboogle	0.200	0.272
Average	0.326	0.320

Table 9 Computational cost for proposer training per iteration.

Method	Rollouts	Wall-Clock Time	GPU-Hours	GPU Util.
HRPO	6 (1 question + 5 predictions)	4.16h	33.28	21.63%
GRPO	20 (4 questions + 16 predictions)	10.58h	84.64	17.84%

Furthermore, we compare the efficiency of HRPO against the standard GRPO algorithm. For GRPO, we generate 4 questions per prompt and perform 4 rollouts per question, totaling 16 solver rollouts per prompt. In contrast, HRPO generates a single question per prompt with 5 solver predictions, reducing the total rollout count to less than one-third of that required by GRPO. We conduct this comparison using the 3B backbone and present the corresponding results in Table 8. The results show that HRPO achieves an average score of 0.326, even surpassing the 0.320 baseline of GRPO. Notably, HRPO demonstrates superior performance on one-hop datasets (e.g., NQ), while GRPO retains an advantage on multi-hop benchmarks (e.g., HotpotQA). This suggests that for complex multi-step reasoning tasks, higher computational resources are still essential to ensure accurate baseline estimation and maximize the learning signals from synthetic data. In summary, HRPO achieves higher aggregate performance while utilizing significantly fewer computational resources than GRPO, highlighting its effectiveness for efficient self-evolution.

Finally, we provide additional significance testing of Dr. Zero against supervised baselines. In Table 1, we follow the `verl` implementation and adopt greedy decoding for evaluation. Here, we perform additional experiments to provide average results with standard deviations. The results are reported in Table 10 and we mark results in bold if the performance gains are *statistically significant*. From the results we observe: (1) for the 3B backbone, Dr. Zero shows consistent gains on knowledge-intensive tasks like NQ. This indicates that self-evolution effectively optimizes search policies for direct retrieval, significantly outperforming the strong Search-R1 baseline. (2) On complex multi-hop datasets, the 7B backbone demonstrates more prominent gains, achieving a significant 7.67% relative improvement on 2WikiMQA. This suggests that larger models are more effective for handling complex interleaved search and reasoning. Overall, we find that our gains are consistent, demonstrating that Dr. Zero can indeed match or exceed supervised baselines.

Generated Question Analysis. To directly examine the structural diversity of questions produced by existing data-free baselines, we sample and analyze 500 generated questions from SQLM* and R-Zero* using `gemini-3-flash-preview` to identify their hop structure. One-hop questions account for 93.4% and 95.6% of their respective outputs, confirming a strong bias toward structurally simple questions. In contrast, Dr. Zero explicitly controls structural coverage through its generation schedule: the default 4:3:2:1 ratio allocates 60% of seed prompts to questions requiring two to four hops. We provide representative Dr. Zero generations at each hop level in Figures 7 to 10. This diagnostic uses diversity specifically to refer to coverage across question hop structures.

Table 10 Significance test of Dr. Zero against supervised methods.

	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle
Qwen2.5-3B-Instruct							
SFT	0.249	0.292	0.104	0.186	0.248	0.044	0.112
R1-Instruct	0.210	0.449	0.171	0.208	0.275	0.060	0.192
Search-R1	0.323	0.537	0.364	0.308	0.336	0.105	0.315
Dr. Zero	0.382 $\pm$.016	0.552 $\pm$.014	0.401 $\pm$.030	0.303 $\pm$.004	0.291 $\pm$.014	0.101 $\pm$.016	0.229 $\pm$.033
Qwen2.5-7B-Instruct							
SFT	0.318	0.354	0.121	0.217	0.259	0.066	0.112
R1-Instruct	0.270	0.537	0.199	0.237	0.292	0.072	0.293
Search-R1	0.397	0.606	0.404	0.380	0.326	0.168	0.408
Dr. Zero	0.400 $\pm$.008	0.599 $\pm$.011	0.405 $\pm$.011	0.362 $\pm$.002	0.351 $\pm$.007	0.112 $\pm$.006	0.352 $\pm$.033

.3 Qualitative Examples

In this section, we present qualitative examples to illustrate the details of the Dr. Zero framework. We begin by providing the full prompts used to guide the proposer and solver in Figure 5 and Figure 6, respectively. To demonstrate the proposer’s ability to synthesize training data of varying complexity, we showcase generation trajectories across different reasoning depths from Figures 7 to 10. These examples highlight the proposer’s capacity to ground questions in initial documents and extend them to more challenging ones via iterative search and reasoning. Finally, Figures 11 to 14 depict the solver’s inference process, verifying its ability to successfully resolve these multi-step queries through structured reasoning and external tool utilization. Note that the last examples for the proposer and solver did not yield satisfactory reasoning or outputs. These instances were specifically selected to illustrate failure modes, such as instruction deviation or truncation due to length constraints. From these qualitative examples, we make the following observations: (1) the proposer effectively scales from single-hop extraction to complex multi-hop synthesis. It demonstrates a sophisticated ability to identify "bridge entities" that link disparate documents, transforming a simple starting point into a challenging multi-step retrieval task. (2) The solver consistently utilizes the thinking blocks to decompose questions into manageable sub-queries. This internal pattern allows the model to verify intermediate facts before proceeding to the next reasoning step, as seen in Figure 13. (3) Both models exhibit high proficiency in using the search tool. The solver, in particular, demonstrates ‘adaptive retrieval’, knowing when its internal knowledge is insufficient and formulating targeted search queries to fill such gaps. (4) While the framework is generally robust, the final examples highlight current limitations in long-context generation. Specifically, as the number of hops increases, the models can occasionally deviate from strict formatting constraints or reach maximum token limits, resulting in incomplete trajectories. Overall, these qualitative examples demonstrate that while the Dr. Zero models are capable of sophisticated multi-hop reasoning, enhancing foundational model capabilities and instruction-following robustness remains essential for extending the frontier of self-evolving LLMs toward increasingly high-complexity tasks.

System
You are Qwen, created by Alibaba Cloud. You are a helpful assistant. # Tools You may call one or more functions to assist with the user query. You are provided with function signatures within <code><tools></tools></code> XML tags: <pre><tools> {"type": "function", "function": {"name": "search", "description": "Searches the web for relevant information based on the given query.", ↪ "parameters": {"type": "object", "properties": {"query_list": {"type": "array", "description": "A list of fully-formed semantic queries. ↪ The tool will return search results for each query.", "enum": null}}, "required": ["query_list"], "strict": false}} </tools></pre> For each function call, return a json object with function name and arguments within <code><tool_call></tool_call></code> XML tags: <pre><tool_call> {"name": <function-name>, "arguments": <args-json-object>} </tool_call></pre>
User / Tool
You are an expert in question generation. Craft one challenging, deterministic question and its single, unambiguous answer based on the ↪ provided source document. The logical path must start from the document and require exactly n hops (i.e., n-1 searches) to reach the ↪ final answer. #### Definitions 1. Hop: A node in the reasoning chain. Hop 1 is the starting entity found in the document. Hop n is the final answer. #### Inputs 1. n: the exact number of hops in the reasoning chain (requiring n-1 searches). 2. Source document: the full source text. #### Process & Tools 1. Analyze the Document and Select the Starting Point - Read and analyze the source document. - Select a specific entity, event or detail explicitly mentioned in the text. This entity becomes Hop 1 (the initial clue). 2. Design the Chain Forwards - From Hop 1 to Hop 2: Identify a factual attribute or relation of Hop 1 that is NOT in the text but can be found via search. The result is ↪ Hop 2. - Iterate: Continue connecting the current Hop i to the next Hop i+1 using deterministic, verifiable relation found via search. - Stop at Hop n: Continue this process until you have exactly n hops. Hop n must be a single, canonical final answer. 3. Reasoning & Search Protocol - Always reason inside <code>`<think> ... </think>`</code> when you plan connections or receive new information. - For each hop transition that requires external information, issue search query using <code>`<tool_call> ... </tool_call>`</code>. - Search results will be provided between <code>`<tool_response> ... </tool_response>`</code> by the system. 4. Output Format - Emit a numbered sequence of EXACTLY n-1 search steps. For each search i (1 to n-1), produce: <pre>`<think> Reasoning step i: Identify Hop i in document/search results, formulate query to reach Hop i+1 </think>` `<tool_call> Query to search Hop i+1 </tool_call>` `[Wait for search results in <tool_response> from system]`</pre> - After completing all searches and arriving at Hop n, output the question and final answer: <pre>`<think> Final reasoning step: Confirm the chain is complete with Hop n and formulate the question </think>` `<question> A challenging question that provides Hop 1 (the initial clue) and asks for the final answer (Hop n) </question>` `<answer> The single, concise final answer (Hop n) </answer>`</pre> #### Examples 1. Example template for Hop n = 1, i.e. no search: <pre>`<think> [Explain how Hop 1 is selected from the source document and how the question is formulated] </think>` `<question> [Question based solely on the text entity Hop 1] </question>` `<answer> [Answer (Hop 1)] </answer>`</pre> 2. Example template for Hop n = 3, i.e. 2 searches: <pre>`<think> [Reasoning step 1: Find Hop 1 in the source document, formulate the query to reach Hop 2] </think>` `<tool_call> [Search query to find Hop 2 based on Hop 1] </tool_call>` `[Wait for search results in <tool_response> from system]` `<think> [Reasoning step 2: Reason on search results to identify Hop 2 and write the next query to find Hop 3] </think>` `<tool_call> [Search query to find Hop 3 based on Hop 2] </tool_call>` `[Wait for search results in <tool_response> from system]` `<think> [Final reasoning step: Confirm Hop 3 in search results and formulate the question starting from Hop 1] </think>` `<question> [Question starting with Hop 1, requiring the solver to find Hop 2 to eventually reach the Answer (Hop 3)] </question>` `<answer> [Answer (Hop 3)] </answer>`</pre> #### Critical Rules 1. Start in Document: Hop 1 must be explicitly present in the source text. Every subsequent hop must be supported by the corresponding ↪ search results. 2. Search is mandatory for n > 1: Each link between hops beyond Hop 1 must use the search engine. 3. Exact search count: Emit exactly (n-1) <code>`<tool_call>`</code> entries, no more, no fewer. 4. No spoilers: The question must mention only Hop 1; do not include or hint at intermediate hops. 5. Clarity: The question is self-contained; the answer is concise and direct (no extra commentary, formatting or explanation). 6. Chain integrity: Each hop must depend strictly on the previous hop. No hop should be skippable or derivable without its immediate ↪ predecessor. Now, generate a question and its answer with n = {hop} hops starting from the following source document: {document}

Figure 5 System prompt and initial instructions for the proposer in Dr. Zero.

System
You are Qwen, created by Alibaba Cloud. You are a helpful assistant. # Tools You may call one or more functions to assist with the user query. You are provided with function signatures within <code><tools></tools></code> XML tags: <pre><tools> {"type": "function", "function": {"name": "search", "description": "Searches the web for relevant information based on the → given query.", "parameters": {"type": "object", "properties": {"query_list": {"type": "array", "description": "A list of → fully-formed semantic queries. The tool will return search results for each query.", "enum": null}}, "required": → ["query_list"]}, "strict": false}} </tools></pre> For each function call, return a json object with function name and arguments within <code><tool_call></tool_call></code> XML tags: <pre><tool_call> {"name": <function-name>, "arguments": <args-json-object>} </tool_call></pre>
User / Tool
Answer the given question. You must conduct reasoning inside <code><think></code> and <code></think></code> first every time you get new → information. After reasoning, if you find you lack some knowledge, you can call a search engine by <code><tool_call> query</code> → <code></tool_call></code> and it will return the top searched results between <code><tool_response></code> and <code></tool_response></code>. You can → search as many times as your want. If you find no further external knowledge needed, you can directly provide the → answer inside <code><answer></code> and <code></answer></code>, without detailed illustrations. For example, <code><answer> Beijing </answer></code>. → Question: {question}

Figure 6 System prompt and initial instructions for the solver in Dr. Zero.

System
System prompt omitted due to page size
User / Tool
Instructions partially omitted due to page size Now, generate a question and its answer with $n = 1$ hops starting from the following source document: (Title: "Robert → Holmes à Court") selling his schoolmates photographs he had taken of them, and by driving them home in exchange for their travel allowances. → Holmes à Court studied forestry at the University of Auckland and Massey University, where he graduated with a → Bachelor of Agricultural Science in 1962...
Assistant
<pre><think> The document mentions that Robert Holmes à Court studied forestry at the University of Auckland and Massey → University. Hop 1 is Robert Holmes à Court. From this, we can deduce that the next step would be to identify which → university he graduated from among these two. </think></pre> <pre><question> At which university did Robert Holmes à Court graduate with a Bachelor of Agricultural Science in 1962? → </question></pre> <pre><answer> Massey University </answer></pre>

Figure 7 Proposer example with $\text{hop} = 1$ and initial document title: Robert Holmes à Court.

System
System prompt omitted due to page size
User / Tool
Instructions partially omitted due to page size Now, generate a question and its answer with $n = 2$ hops starting from the following source document: (Title: "Equivalent carbon content") Society adopted the critical metal parameter (Pcm) for weld cracking, which was based on the work from Ito and Bessyo, is: ↪ If some of the values are not available, the following formula is sometimes used: The carbon equivalent is a measure of ↪ the tendency of the weld to form martensite on cooling and to suffer brittle fracture. When the carbon equivalent is ↪ between 0.40 and 0.60 weld preheat may be necessary...
Assistant
<think> The source document mentions the "carbon equivalent" as a measure of the tendency of welds to form martensite and suffer brittle fracture. Hop 1 is "carbon equivalent". For Hop 2, the formula to determine if a spot weld will fail ↪ needs to be found, which logically follows from the concept of carbon equivalent.</think> <tool_call> <pre>{ "name": "search", "arguments": { "query_list": ["carbon equivalent formula to determine spot weld failure"] } }</pre> </tool_call>
User / Tool
<tool_response> Search results omitted due to page size </tool_response>
Assistant
<think> Reviewing the search results, Doc 1 directly provides the formula used to determine if a spot weld will fail, which is ↪ referenced in the source document. The formula is: The carbon equivalent formula is used to determine if a spot weld ↪ will fail. Hop 1 is "carbon equivalent", and Hop 2, which is the final answer, is the formula used to determine if a spot ↪ weld will fail.</think> <question> What specific formula is used to determine if a spot weld will fail based on the carbon equivalent? </question> <answer> The carbon equivalent formula </answer>

Figure 8 Proposer example with $\text{hop} = 2$ and initial document title: Equivalent carbon content.

System
System prompt omitted due to page size
User / Tool
Instructions partially omitted due to page size
Now, generate a question and its answer with $n = 3$ hops starting from the following source document: (Title: "Steven ↪ Febey") selected in the national draft. He fell just short of Robert Flower's club record of 272 games and as of 2007 only four people ↪ have played more games for the Demons After the 2002 season, Febey went to Bali with his teammates and was caught ↪ up in the bombings which devastated the island. Standing beside former Demons player Steven Armstrong and David ↪ Robbins, Febey was about to enter the Sari Club when the car bomb exploded...
Assistant
<think> Reasoning step 1: Identify Hop 1 in the document, formulate the query to reach Hop 2 </think> <tool_call> {"name": "search", "arguments": {"query_list": ["other players who played in both 1988 and 2000 AFL grand finals for ↪ Melbourne Demons"]}} </tool_call>
User / Tool
<tool_response> Search results omitted due to page size </tool_response>
Assistant
<think> Hop 2 in the document and search results is not explicitly mentioned, so we need to further refine the search. The ↪ document mentions David Neitz, the captain of Melbourne, and players like Shane Woewodin and Greg Healy. However, ↪ the search results point us to the 1988 and 2000 AFL Grand Finals, indicating that we need to look for a player ↪ specifically from these finals who also played for Melbourne. The final hop, or Hop 3, must be a well-known player from ↪ the Melbourne Demons who played in both 1988 and 2000 grand finals, which can be Robert Harvey based on historical ↪ knowledge. </think> <think> Reasoning step 2: Identify Hop 2 in the search results and reformulate to find Hop 3 </think> <tool_call> {"name": "search", "arguments": {"query_list": ["Robert Harvey Melbourne Demons 1988 2000 grand finals"]}} </tool_call>
User / Tool
<tool_response> Search results omitted due to page size </tool_response>
Assistant
<think> The search results confirm that Robert Harvey played for the Melbourne Demons in both 1988 and 2000, making ↪ him the well-known player who fits the criteria. Therefore, Hop 3 is "Robert Harvey". The question is formulated to ↪ start with the explicit entity in the document, "Steven Febey", and lead to the single, unambiguous answer, "Robert ↪ Harvey". </think> <question> Who is the only other active Melbourne Demons player to have played in both the 1988 and 2000 AFL Grand ↪ Finals, alongside Steven Febey? </question> <answer> Robert Harvey </answer>

Figure 9 Proposer example with $\text{hop} = 3$ and initial document title: Steven Febey.

System
System prompt omitted due to page size
User / Tool
Instructions partially omitted due to page size
Now, generate a question and its answer with $n = 4$ hops starting from the following source document: (Title: "Australian Labor Party (Australian Capital Territory Branch)") ↪ within the ALP. The issue was Vietnam and it involved a North-South split. The contender Gordon Walsh won the ↪ preselection, but this was later overturned by the NSW Branch...
Assistant
<think> Reasoning step 1: Identify Hop 1 in the document, which is the 'Canberra City (North) Branch'. It mentions 118 ↪ members in 1968 but does not directly relate to the ACT Branch establishment. Formulate a query to find information ↪ about the ACT Branch's autonomy. </think> <tool_call> {"name": "search", "arguments": {"query_list": ["when did the alp national conference establish the self-governing act ↪ branch"]}} </tool_call>
User / Tool
<tool_response> Search results omitted due to page size </tool_response>
Assistant
<think> Reasoning step 2: From the search result in Document 1, it is mentioned that "ALP National Conference ↪ established an autonomous ACT Branch in 1973". This provides us with our Hop 2, which is 1973. Now, I need to use ↪ the information from Hop 2 (1973) to formulate a query for Hop 3. Since the information moves from the establishment ↪ of the ACT Branch to the self-governance of the ACT, a query about when the ACT became a self-governing territory ↪ would be appropriate. </think> <tool_call> {"name": "search", "arguments": {"query_list": ["when did the australian capital territory become a self-governing ↪ territory"]}} </tool_call>
User / Tool
<tool_response> Search results omitted due to page size </tool_response>
Assistant
<think> Reasoning step 3: From the search results, it is clear that the ACT became a self-governing territory in 1989. This ↪ gives us Hop 3, which is 1989. The final step is to create a query that determines the final answer (Hop 4) based on ↪ these previous hops. </think> <tool_call> {"name": "search", "arguments": {"query_list": ["when did the australian capital territory Legislative Assembly first sit"]}} </tool_call>
User / Tool
<tool_response> Search results omitted due to page size </tool_response>
Assistant
<think> Reasoning step 4: The search results clearly indicate that the Australian Capital Territory Legislative Assembly ↪ first sat in 1989. This is our final answer, Hop 4, which directly follows from the previous hops. The challenge in the ↪ question is designed such that the initial entity 'Canberra City (North) Branch' is provided as the starting point, and ↪ the solver needs to follow the hops as established to reach the final answer. Now I will formulate the question and the final answer: </think> <question> In which year did the Australian Labor Party (ACT Branch) first sit as the Legislative Assembly of the ↪ Australian Capital Territory? </question> <answer> 1989 </answer>

Figure 10 Proposer example with $\text{hop} = 4$ and initial document title: Australian Labor Party (Australian Capital Territory Branch).

System
System prompt omitted due to page size
User / Tool
Answer the given question. You must conduct reasoning inside <code><think></code> and <code></think></code> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <code><tool_call> query</code> <code></tool_call></code> and it will return the top searched results between <code><tool_response></code> and <code></tool_response></code>. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside <code><answer></code> and <code></answer></code>, without detailed illustrations. For example, <code><answer> Beijing </answer></code>. ↪ Question: Where was the place of burial of Charles Mathew's father?
Assistant
I need to conduct a search to find out where Charles Mathew's father was buried. <pre><tool_call> {"name": "search", "arguments": {"query_list": ["Where was the place of burial of Charles Mathew's father"]}}</pre> </tool_call>
User / Tool
<pre><tool_response> Search results omitted due to page size </tool_response></pre>
Assistant
According to the search results, Father Mathew, Charles Mathew's father, was buried in Cork city in a cemetery which he ↪ had himself established. <answer>Cork</answer>

Figure 11 Solver example for the question: Where was the place of burial of Charles Mathew’s father?

System
System prompt omitted due to page size
User / Tool
Answer the given question. You must conduct reasoning inside <code><think></code> and <code></think></code> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <code><tool_call> query</code> and it will return the top searched results between <code><tool_response></code> and <code></tool_response></code>. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside <code><answer></code> and <code></answer></code>, without detailed illustrations. For example, <code><answer> Beijing </answer></code>. ↪ Question: Where was the husband of Octavie Coudreau born?
Assistant
I need to conduct a search to find out where the husband of Octavie Coudreau was born. <code><tool_call></code> <code>{"name": "search", "arguments": {"query_list": ["Where was the husband of Octavie Coudreau born"]}}</code> <code></tool_call></code>
User / Tool
<code><tool_response></code> Search results omitted due to page size <code></tool_response></code>
Assistant
According to the search results, the husband of Octavie Coudreau was Henri Coudreau. Further searching for "Where was ↪ Henri Coudreau born" is needed. <code><tool_call></code> <code>{"name": "search", "arguments": {"query_list": ["Where was Henri Coudreau born"]}}</code> <code></tool_call></code>
User / Tool
<code><tool_response></code> Search results omitted due to page size <code></tool_response></code>
Assistant
According to the search results, Henri Coudreau was born in Sonnac. Therefore, the husband of Octavie Coudreau was born ↪ in Sonnac. <code><answer>Sonnac</answer></code>

Figure 12 Solver example for the question: Where was the husband of Octavie Coudreau born?

System
System prompt omitted due to page size
User / Tool
Answer the given question. You must conduct reasoning inside <code><think></code> and <code></think></code> first every time you get new ↪ information. After reasoning, if you find you lack some knowledge, you can call a search engine by <code><tool_call></code> query ↪ <code></tool_call></code> and it will return the top searched results between <code><tool_response></code> and <code></tool_response></code>. You can ↪ search as many times as your want. If you find no further external knowledge needed, you can directly provide the ↪ answer inside <code><answer></code> and <code></answer></code>, without detailed illustrations. For example, <code><answer></code> Beijing <code></answer></code>. ↪ Question: Which film has the director who died earlier, Gold For The Caesars or Heinz In The Moon?
Assistant
I need to find out the directors of the films 'Gold For The Caesars' and 'Heinz In The Moon' to determine which one was ↪ directed by the director who died earlier. <code><tool_call></code> <pre>{"name": "search", "arguments": {"query_list": ["which film has the director who died earlier, Gold For The Caesars or Heinz In The Moon"]}}</pre> <code></tool_call></code>
User / Tool
<code><tool_response></code> Search results omitted due to page size <code></tool_response></code>
Assistant
From the search results, it is clear that the film 'Gold for the Caesars' was directed by Andrés De Toth in the United States ↪ and both De Toth and Sabatino Ciuffini in Italy. However, the information does not specify when Andrés De Toth died. Let's now find out the director of 'Heinz In The Moon'. <code><tool_call></code> <pre>{"name": "search", "arguments": {"query_list": ["director of Heinz In The Moon"]}}</pre> <code></tool_call></code>
User / Tool
<code><tool_response></code> Search results omitted due to page size <code></tool_response></code>
Assistant
The search results indicate that the director of 'Heinz In The Moon' was Melvin Bernhardt, who died on September 12, 2015. Now, let's find out when Andrés De Toth died. <code><tool_call></code> <pre>{"name": "search", "arguments": {"query_list": ["Andrés De Toth died"]}}</pre> <code></tool_call></code>
User / Tool
<code><tool_response></code> Search results omitted due to page size <code></tool_response></code>
Assistant
The search results indicate that Andrés De Toth died on November 28, 2006. Since Melvin Bernhardt died in 2015 and ↪ Andrés De Toth died in 2006, Andrés De Toth died earlier. Therefore, the film 'Gold for the Caesars' was directed by the director who died earlier. <code><answer></code> Gold for the Caesars <code></answer></code>

Figure 13 Solver example for the question: Which film has the director who died earlier, Gold For The Caesars or Heinz In The Moon?

System
System prompt omitted due to page size
User / Tool
Answer the given question. You must conduct reasoning inside <code><think></code> and <code></think></code> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <code><tool_call></code> query <code></tool_call></code> and it will return the top searched results between <code><tool_response></code> and <code></tool_response></code>. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside <code><answer></code> and <code></answer></code>, without detailed illustrations. For example, <code><answer></code> Beijing <code></answer></code>. Question: When did the people who first imported slaves to New York, come to the country where Lord of the Rings was filmed?
Assistant
<pre><tool_call> {"name": "search", "arguments": {"query_list": ["when did the people who first imported slaves to New York come to the country where Lord of the Rings was filmed"]}} </tool_call></pre>
User / Tool
<pre><tool_response> Search results omitted due to page size </tool_response></pre>
Assistant
Based on the search results, the first slaves were imported to New York in 1626 by the Dutch West India Company. The last slaves were freed on July 4, 1827. Now, I need to find out where Lord of the Rings was filmed. Let's search for that information. <pre><tool_call> {"name": "search", "arguments": {"query_list": ["where was lord of the rings filmed"]}} </tool_call></pre>
User / Tool
<pre><tool_response> Search results omitted due to page size </tool_response></pre>
Assistant
The Lord of the Rings film series was primarily filmed in New Zealand. To answer the question, we need to find out when people came to New Zealand. Let's search for the history of immigration to New Zealand. <pre><tool_call> {"name": "search", "arguments": {"query_list": ["history of immigration to New Zealand"]}} </tool_call></pre>
User / Tool
<pre><tool_response> Search results omitted due to page size </tool_response></pre>
Assistant
Based on the search results, the first people to settle in New Zealand were Polynesians...

Figure 14 Solver example for the question: When did the people who first imported slaves to New York, come to the country where Lord of the Rings was filmed?